

NSS College of Engineering Palakkad
Department of Computer Science and Engineering

Software Engineering Micro Project Report

Course Code: PECST411
Course Name: Software Engineering
Programme: B.Tech Computer Science and Engineering
Semester: S4

Project Title:

Digital Farmer Assistance System

Team Members:

1. [Name 1]
2. [Name 2]
3. [Name 3]
4. [Name 4]
5. [Name 5]

Batch: A / B

Submission Date: _____

Contents

Abstract	6
1 Introduction	7
1.1 Background	7
1.2 Objectives	7
1.3 Scope	8
2 Problem Statement	9
2.1 Initial Problem Statement	9
2.2 Refined Problem Statement	10
3 Requirement Analysis	10
3.1 Functional Requirements	10
4 Feasibility Study	11
4.1 Technical Feasibility	11
4.2 Economic Feasibility	11
4.3 Conclusion	13
5 Software Requirement Specification (SRS)	13
5.1 System Overview	13
5.2 System Features	13
5.3 Assumptions	14
5.4 Constraints	14
6 System Design	15
6.1 Architectural Overview	15
6.2 Architecture Diagram	15
6.3 Module Design	16
6.4 Cohesion Analysis	16
6.5 Coupling Analysis	17
7 UML Diagrams	17
7.1 Use Case Diagram	17
7.2 Class Diagram	19
8 Coding Strategy	20
8.1 Programming Language environment	20
8.2 Tools Used	20
8.3 Project Architecture Structure	21

8.4	Coding Standards	22
8.5	Representative Implementation Code	22
9	Testing Strategy	23
9.1	Testing Approach	23
9.2	Functional Test Cases	23
9.3	Boundary and Negative Test Cases	25
10	Deployment Plan	25
10.1	Deployment Environment	25
10.2	Local Deployment Steps	26
10.3	Cloud Deployment Implementation	26
10.4	Ongoing System Maintenance	27
11	Reflection and Learning Outcomes	27
11.1	Challenges faced during client interaction	27
11.2	Lessons learned	28
12	References	28
	References	28
13	Appendix A – Product Screenshots	29

List of Figures

1	Four-Layer Architecture of the System	15
2	System Use Case Diagram	18
3	System Class Diagram	19
4	Farmer Login Interface: Mobile-first phone number and password authentication form.	29
5	Admin Login Interface: Email and password form mapped exclusively for administrator user access tracking.	29
6	Farmer Dashboard: Unified data stream rendering specific crop advisories, an integrated 5-day weather forecast, and local market price summary cards.	29
7	Market Prices Module: Searchable and highly filterable data table rendering recent crop prices tagged with the active date and source market coordinates.	30
8	Weather Module: Rendered 5-day localized weather sequence showcasing the automated alert banners dynamically triggered by external extreme weather metric flags.	30
9	Crop Advisory Module: Rendered layout featuring season-filtered advisory content dynamically leveraging the application’s native multilingual database translation labels.	30
10	Query Submission Form: Interactive module utilized by farmers to directly submit agricultural issue reports explicitly requesting expert administrative resolutions.	30
11	Admin Panel Architecture: Backend CRUD (Create, Read, Update, Delete) portal exclusively used by administrative profiles to constantly manage advisory flows and market price points.	31

List of Tables

1	Functional Requirements of the Digital Farmer Assistance System	10
2	System Module Design Mapping	16
3	Primary Third-Party Dependencies	21
4	System Functional Test Cases	24
5	Boundary and Negative Scenario Emulations	25

Abstract

The Digital Farmer Assistance System addresses the critical information gap faced by small-scale farmers in rural areas by providing a centralized platform for essential agricultural data. Traditional farming operations often suffer from poor resource management due to inaccessible or delayed market trends, volatile weather patterns, and unverified crop guidance. This project presents a web-based portal developed to deliver real-time, validated crop-specific agricultural advisories, local market price updates, and a five-day localized weather forecasting mechanism integrated with extreme weather alerts. Built on a four-tier architecture featuring a Node.js runtime, an Express.js server, and a MySQL database backend, the system emphasizes scalability and low-latency access over varied bandwidths. The frontend provides a mobile-first user interface tailored to accessibility constraints. The implementation features a role-based access control paradigm separating farmer and administrative interfaces, ensuring secure and private user interactions. A core component of the system is the expert query resolution module, bridging the gap between agronomists and farmers. Functionalities including multilingual database-level support and robust offline fallback database pathways cater particularly to communities with disparate levels of digital literacy and unreliable connectivity. As a result, the platform aims to empower farmers through data-driven decisions that will simultaneously improve crop yields and promote sustained financial stability.

1. Introduction

1.1 Background

The agricultural sector remains a cornerstone of the Indian economy, employing a substantial portion of the rural population and contributing significantly to the national gross domestic product. Despite its critical importance, the sector is plagued by systemic inefficiencies, primarily stemming from a profound information asymmetry. Small-scale farmers, who constitute the vast majority of the agricultural workforce, frequently operate in information silos. They are largely disconnected from real-time data regarding optimal farming practices, volatile market price fluctuations, and rapidly changing micro-climatic conditions.

This digital divide in rural areas exacerbates the vulnerability of farmers. Traditional decision-making processes rely heavily on anecdotal experience or delayed information cascaded through informal networks. Consequently, crop yields are often suboptimal due to misaligned planting schedules, unmanaged pest outbreaks, or inappropriate fertilizer application. Furthermore, the lack of transparent, real-time access to regional market prices leaves farmers susceptible to exploitation by intermediaries, resulting in diminished financial returns. Compounding these challenges is the increasing unpredictability of weather patterns driven by global climate change. Without highly localized and timely weather forecasts, farmers are unable to take preemptive measures against extreme weather events such as sudden torrential rains or prolonged dry spells, often leading to devastating crop losses. The integration of modern information and communication technologies (ICT) into the agricultural workflow is therefore not just beneficial, but an urgent necessity to bridge this information gap and foster sustainable agricultural development.

1.2 Objectives

The primary objective of this project is to architect and deploy a comprehensive, accessible digital ecosystem tailored to the needs of the modern farmer. Specifically, the system aims to:

1. Provide real-time, validated crop-specific agricultural advisories that are dynamically tailored to the current growth stage of the crop and the prevailing agricultural season.
2. Deliver live, daily-updated market price information sourced directly from regional and local markets to empower farmers during financial negotiations and sales planning.
3. Offer highly localized, five-day weather forecasts integrated with automated alert mechanisms to warn users of impending extreme weather conditions, thereby enabling proactive crop protection.
4. Enable an interactive query resolution module, allowing farmers to submit specific agricultural problems or photographs of crop diseases directly to agricultural experts for personalized guidance.
5. Support broad accessibility through a mobile-first, multilingual interface designed

specifically for rural user bases with varying levels of digital literacy and potentially unstable internet connectivity.

1.3 Scope

The scope of the Digital Farmer Assistance System encompasses a fully functional, web-based platform optimized for mobile access. The system is engineered around two primary user roles: the Farmer and the Administrator (inclusive of a Superadmin sub-role for privileged platform configuration).

From a technical perspective, the application is developed using a modern web development stack consisting of a Node.js runtime environment, Express.js for the backend server architecture, and a robust MySQL 8.0 database for persistent data storage. The frontend is constructed using responsive HTML, CSS, and Vanilla JavaScript to ensure minimal load times on low-bandwidth networks.

A critical component of the system's scope is its integration with external services, specifically the OpenWeatherMap API, to fetch live meteorological data. Importantly, the scope includes a sophisticated database fallback strategy to ensure continued operational capability even when external API connectivity is interrupted. Additionally, the system provides integrated multilingual support implemented directly at the database level, allowing advisory content and interface elements to be served in the user's preferred regional language without necessitating complex application-layer internationalization frameworks.

Report Organisation The remainder of this document is organized to systematically detail the software engineering lifecycle of the project. Chapter 3 critically evaluates the initial problem statement, identifying ambiguities and presenting a refined formulation. Chapter 4 provides a comprehensive requirement analysis, delineating both functional and non-functional requirements alongside an assessment of the project's alignment with the United Nations Sustainable Development Goals. Chapter 5 discusses the multi-dimensional feasibility study undertaken prior to project commencement. Chapter 6 presents the formal Software Requirement Specification (SRS). Chapter 7 explores the architectural and system design, including the rationale for the selected software patterns. Chapter 8 visualizes the system's behavioral and structural models through UML diagrams. Chapter 9 outlines the adopted coding strategy and environment setup. Chapter 10 details the comprehensive testing strategy and specific test cases executed. Chapter 11 provides a concrete deployment plan for transitioning the application to a production environment. Finally, Chapter 12 offers a reflective analysis of the challenges encountered and the learning outcomes achieved throughout the project development lifecycle.

2. Problem Statement

2.1 Initial Problem Statement

Farmers currently lack timely and reliable access to localised crop advisory, real-time market prices, and critical weather information. This profound information gap results in inefficient farming practices, significantly reduced crop yields, and increased financial vulnerability due to unpredictable market volatility and increasingly erratic weather patterns. Without a centralized, accessible platform to consolidate this disparate data, decision-making remains reactive rather than data-driven, severely limiting the economic mobility and operational resilience of small-scale agricultural communities.

Ambiguities, Inconsistencies, and Incompleteness Identified Upon critical evaluation of the initial problem statement, several substantial flaws and areas requiring clarification were identified. Formalizing the software requirements necessitated a thorough deconstruction of these issues.

Ambiguities

- The phrase “*timely access*” is overly subjective and lacks measurable criteria. It does not specify whether the updates should be provided in real-time (instantaneously), on an hourly basis, or via a daily batch process.
- The term “*market prices*” fails to define the geographic scope of the data. It is ambiguous whether the system should aggregate national commodity exchange rates, state-level averages, or prices from highly localized regional *mandis* (markets).

Inconsistencies

- The initial statement emphasizes “*reliable access*” but provides no mechanism for how the data (especially crowdsourced or admin-entered advisories) will be vetted, validated, or maintained for data integrity.
- There is a contradiction involving the target demographic: asserting the need for constant access to live data while implicitly serving rural areas known for intermittent and unreliable internet connectivity. No offline or low-bandwidth strategy was explicitly detailed.
- Although the system aims to improve farmers’ financial standing, there is no mention of the secure handling of their personal and geographic profiles, leading to inconsistent application of data privacy standards.

Incompleteness

- The requirement for “*weather information*” does not specify the delivery mechanism—whether alerts require the user to actively check the platform or if the system

will passively push automated warnings to the user during extreme meteorological events.

- The problem statement ignores the multilingual necessity of the target audience. Presenting complex agricultural data solely in English would render the platform unusable for the majority of the intended user base.

2.2 Refined Problem Statement

Addressing the identified shortcomings, the problem statement was systematically restructured to provide clear, actionable, and measurable engineering objectives.

Refined Problem Statement: To design, develop, and deploy a secure, web-based Digital Farmer Assistance System that centralises disparate agricultural data streams. The system must provide:

- Real-time synchronisation of localized regional market prices and five-day weather forecasts (augmented by a database fallback mechanism for low-connectivity scenarios).
- Curated, validated, and season-specific crop advisories delivered through a mobile-first, multilingual user interface tailorable to varying levels of digital literacy.
- An interactive query resolution pipeline connecting farmers with administrative agricultural experts.

Goal: To systematically eliminate information asymmetry within the rural agricultural sector, thereby allowing operators to execute data-driven, preemptive decisions that measurably enhance crop productivity and ensure financial sustainability within constrained, low-bandwidth environments.

3. Requirement Analysis

3.1 Functional Requirements

Functional requirements represent the specific behavioral functions that the system must perform to satisfy the user's needs. The core capabilities of the Digital Farmer Assistance System are explicitly defined to track user interactions and data processing workflows.

Table 1: Functional Requirements of the Digital Farmer Assistance System

FR ID	Description
FR1	User Management: The system shall allow farmers to securely register, authenticate, and manage personalized profiles, including the selection of preferred crop types and geographic regions.
FR2	Advisory Engine: The system shall dynamically filter and provide crop-specific agricultural guidance based on the inputted growth stage, prevailing season, and the farmer's declared preferences.

Continued on next page

Table 1 – *Continued from previous page*

FR ID	Description
FR3	Information Retrieval: The system shall aggregate and display daily updated market prices from various regional markets, alongside localized 5-day weather forecasts utilizing external API integrations.
FR4	Alert System: The system shall continuously monitor incoming meteorological data and push automated notifications or visual highlights on the dashboard for extreme weather warnings.
FR5	Admin Control: The system shall provide a comprehensive, secure dashboard for administrators to perform CRUD (Create, Read, Update, Delete) operations on market prices, weather data, and advisory content.
FR6	Interaction Module: The system shall encompass a dedicated query resolution module allowing farmers to submit textual descriptions of agricultural issues to registered experts for tailored responses.

4. Feasibility Study

4.1 Technical Feasibility

Technical feasibility assesses whether the proposed system can be practically implemented using currently accessible technologies, given the defined constraints and performance criteria. The Digital Farmer Assistance System is highly technically feasible. The entire back-end architecture is heavily reliant on mature, battle-tested web technologies, primarily a Node.js runtime operating an Express.js server, communicating with a MySQL 8.0 relational database. This combination ensures high throughput and stability for concurrent I/O operations. Furthermore, the integration capabilities required for third-party weather data via the OpenWeatherMap REST API are straightforward, operating well within the provider's free-tier limitations of 60 calls per minute. The required multilingual capabilities do not necessitate complex external localization libraries; instead, they are efficiently managed natively within the MySQL database through an indexed language attribute within the ADVISORY table. Essential security protocols, including JSON Web Token (JWT) authentication for stateless session management and bcrypt for cryptographic password hashing, are readily implemented via established Node.js modules. The architecture intentionally avoids hardware dependencies outside of standard client devices and omits unproven experimental AI models, thereby minimizing technical complexity risks. Finally, a robust fallback caching paradigm mitigates the risk of external API downtime, cementing the overarching technical viability of the platform.

4.2 Economic Feasibility

Economic feasibility evaluates the cost-benefit ratio associated with the development, deployment, and long-term maintenance of the software compared against the tangible finan-

cial improvements provided to the end-users. The underlying infrastructure heavily favors a low-cost implementation. The entire technology stack—Node.js, Express, and MySQL—is open-source, resulting in zero initial or recurring proprietary licensing expenditures. Deployment and hosting can be seamlessly managed through cost-effective cloud platform as a service (PaaS) providers such as Railway or Render, which offer free prototype tiers and inexpensive scaling pathways. From the user’s perspective, hardware acquisition costs are negligible; interaction requires only a standard smartphone with a basic web browser and modest internet connectivity, removing the financial barriers typically associated with high-end precision agricultural tools. The return on investment (ROI) is substantial: farmers are empowered to optimize crop yields through tailored advisory support, maximize profit margins via predictive market timing based on live price metrics, and radically reduce financial losses through preemptive interventions following automated extreme weather alerts. Consequently, the project strongly justifies its low inherent development and operational costs by delivering massive downstream economic impact.

Operational Feasibility Operational feasibility determines the ease with which the proposed system can be integrated into the daily workflows of its target demographic, acknowledging their specific socio-technical constraints. The platform is intentionally engineered with a mobile-first user interface tailored explicitly for rural operators who may exhibit varying degrees of digital literacy. The intuitive, high-contrast visual design guarantees that critical metrics—such as volatile market prices or severe weather alerts—are instantly comprehensible without demanding sophisticated technical navigation. Operationally, the software utilizes an unambiguous role-based access control paradigm. Farmers solely interact with the consumption modules (market data, advisory content, weather metrics, and query submission), ensuring an uncluttered, focused cognitive load. Conversely, administrative personnel interact with specialized CRUD interfaces optimized for efficient content management and verification. Additionally, the onboarding friction for new rural users is effectively minimized by utilizing their familiar primary identifier—their mobile phone number—in conjunction with a secure password, negating the requirement for cumbersome email-centric registration workflows. The operational integration of the system is thus both simple and culturally congruent.

Time Feasibility Schedule feasibility analyzes the probability that the final software product can be successfully designed, engineered, tested, and deployed within the strict designated timeframe available for the S4 semester Software Engineering Micro Project. The project is confirmed to be schedule-feasible. The architectural decomposition of the system into completely independent modules (e.g., `routes/auth.js`, `routes/market.js`, `routes/weather.js`, `routes/advisory.js`) acts as a catalyst for parallel development execution. Distinct teams or developers can concurrently build and isolate their respective components without blocking dependencies. The standardized RESTful API contracts allow frontend and backend implementations to proceed simultaneously. Furthermore, individual

modules can be rigorously unit-tested in isolation well before comprehensive system integration commences, allowing schedule-threatening bugs to be isolated and resolved early in the SDLC. The software's low-coupling foundation ensures that any future requirements or unforeseen pivot modifications do not necessitate total system re-architecting, providing a vital buffer against schedule overruns.

4.3 Conclusion

Drawing upon continuous evaluation across technical, economic, operational, and scheduling dimensions, it is definitively concluded that the implementation of the Digital Farmer Assistance System is entirely feasible. The project's constraints are thoroughly managed through strategic technology choices, resulting in a sustainable, high-impact digital platform capable of revolutionizing rural agricultural operations.

5. Software Requirement Specification (SRS)

5.1 System Overview

The Digital Farmer Assistance System operates as an integrated web-based platform specifically designed to deliver vital agricultural support mechanisms to rural farmers. The software centralizes critical datasets, offering a cohesive portal providing three primary services: dynamic crop advisory mechanisms, verified regional market pricing tracking, and localized meteorological forecasting. The ecosystem is meticulously administered and curated by a dedicated content management team composed of agricultural experts operating via a specialized secure admin control panel. The system effectively mitigates the rural information deficit by providing an accessible, mobile-first centralized dashboard engineered specifically for low-bandwidth cellular environments.

5.2 System Features

The software's functional scope is demarcated into distinct features, detailing operational capabilities provided to the designated users.

1. **User Management and Preferences (FR1):** The platform encompasses a comprehensive registration and authentication workflow. Farmers can create, read, update, and manage customizable profiles, designating their primary crop portfolios and operational geographic regions. This profile metadata forms the foundational filter mechanism dictating personalized data aggregation throughout the remainder of the application.
2. **Dynamic Advisory Engine (FR2):** The system incorporates a business-logic module that parses a farmer's crop selections and cross-references them with the current seasonal timeline. This allows the system to autonomously filter and present highly specific agricultural advisories, ensuring rural users are not inundated with irrelevant or out-of-season guidance.

3. **Aggregated Information Retrieval (FR3):** A unified dashboard module autonomously aggregates data from two disparate streams: live regional market prices provided and updated by administrative personnel, alongside a comprehensive, localized 5-day weather forecast ingested automatically through secure API integrations.
4. **Automated Alert and Notification System (FR4):** The application continuously monitors incoming meteorological data for predetermined critical thresholds. Should extreme weather conditions be predicted, the system generates automated warnings directly to the user's dashboard interface, highlighting critical operational risks for the farmer.
5. **Administrative Control Dashboard (FR5):** Privileged users with administrator roles are provided an intuitive backend portal. This interface allows for the rapid creation, modification, and deletion (CRUD functions) of all system content, including raw market prices, static advisories, and overriding fallback weather data.
6. **Expert Interaction Module (FR6):** To solve unique agricultural challenges not addressed by static advisories, a bidirectional query interface allows farmers to submit free-text questions. Specialized administrative users act as subject matter experts, directly responding to and resolving farmer inquiries through a managed ticket queue system.

5.3 Assumptions

The successful execution and operation of the software architecture are predicated upon several foundational assumptions regarding user environments and technical dependencies.

1. Users will operate within environments with at least intermittent, low-bandwidth internet connectivity. Total offline scenarios rely strictly on previously cached database fallback mechanisms but necessitate occasional synchronization.
2. The hosting environment will consistently maintain a standard Node.js (v18+) runtime ecosystem and an operational MySQL (v8.0+) relational database instance.
3. A root 'Superadmin' profile must be securely seeded into the raw database prior to deployment to enable the initial platform administration and subsequent staff provisioning.
4. While the external Weather API key is optimal for live forecasting, its absence is not catastrophic; the system assumes a highly functional offline or static demonstration mode is permanently configured.

5.4 Constraints

Constraints define structural boundaries that limit developmental options or restrict operational methodologies.

1. The software scope is strictly confined to a web-based architecture. The development of distinct native iOS or Android mobile applications is explicitly excluded from the current phase.

2. Live API connectivity is implemented solely for meteorological data. Regional market pricing data remains a manual dependency, structurally requiring dedicated administrative curation to maintain accuracy.
3. Advanced telecommunication integration, such as native SMS push notifications or backend mobile alerts, is marked exclusively as future system extensions. Alerts operate structurally at the application level only.
4. Native software localization mechanics (e.g., i18n libraries) are not utilized. Language support is explicitly bounded by the manually seeded multilingual content existing within the database fields.

6. System Design

6.1 Architectural Overview

The Digital Farmer Assistance System purposefully adopts a formal **four-layer software architecture**, establishing an explicit separation of computing concerns throughout the application stack. This structural design isolates the user interface semantics from the volatile business logic and raw data persistence layers. This separation fundamentally guarantees that alterations to the frontend HTML layouts or database schema structures will not necessitate cascading, breaking changes across the entire monolithic codebase. This architecture ultimately facilitates cleaner unit testability, highly streamlined debugging, and vastly superior post-deployment maintenance.

6.2 Architecture Diagram

The functional flow of data between the four primary operational layers is visually formalized via the following structured architectural diagram. Top-down requests initialize logic rendering, while bottom-up cascades fulfill data persistence queries.

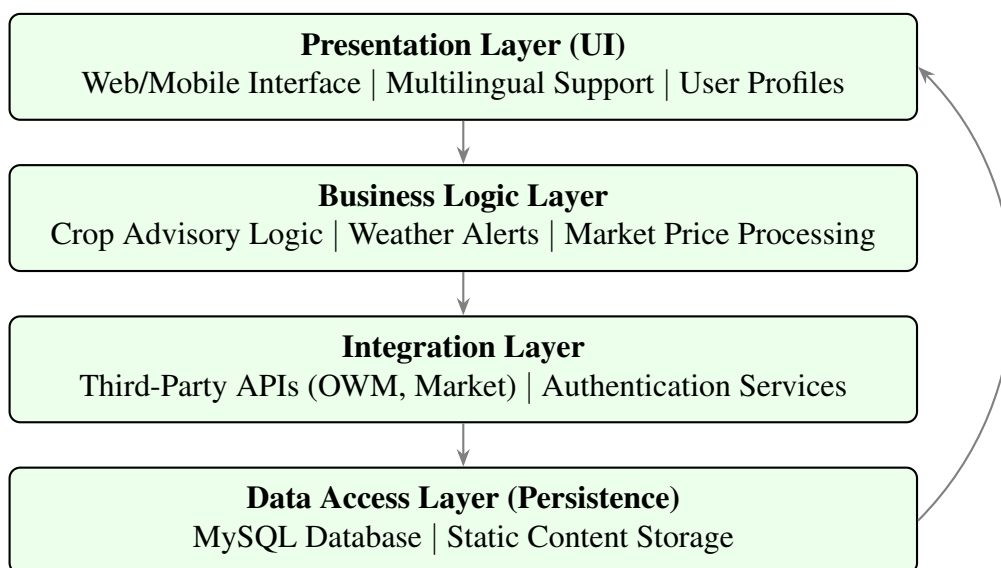


Figure 1: Four-Layer Architecture of the System

6.3 Module Design

The system logically divides complex backend requirements into distinct, deployable modules, mapped explicitly to discrete implementation files in the codebase.

Table 2: System Module Design Mapping

Layer	Module	Implementation File
Presentation	User Interface Module	public/index.html
Presentation	Multilingual Support Module	DB language attribute + farmer preference config
Business Logic	User Management Module	routes/auth.js, middleware/auth.js
Business Logic	Advisory Engine Module	routes/advisory.js
Business Logic	Alert Manager Module	is_extreme flag in WEATHER_FORECAST, routes/weather.js
Integration	Market Price Integration Module	routes/market.js
Integration	Weather Service Module	routes/weather.js (OWM API → DB → mock)
Data Access	Data Persistence Module	db.js (mysql2 connection pool)
Data Access	Content Management Module	Admin CRUD controllers across all route files

Module Description Summary: The Presentation Layer dictates responsive HTML layouts and processes DB translation strings. The Business Logic layer contains primary logic controllers determining weather alert calculations and advisory payload generation. The Integration layer contains networking abstractions specifically focused on third-party API processing and JWT validation mechanisms. Finally, the Data Access Layer universally handles optimized SQL query execution directly to the MySQL cluster.

6.4 Cohesion Analysis

Cohesion evaluates the degree to which elements inside an individual module belong logically together. The Digital Farmer Assistance System actively enforces functional cohesion, widely regarded as the strongest and most desirable form. The User Management Module is strictly confined to executing authentication strategies and profile mutations; it possesses no capability to process structural advisories. The Weather and Alert module exclusively computes meteorological data and alert algorithms without manipulating agricultural content. This intense functional isolation guarantees vastly superior code readability and ensures individual modules can be autonomously maintained or rewritten entirely without unintended global behavioral degradation.

6.5 Coupling Analysis

Coupling measures the degree of functional interdependence or data reliance between discrete modules. Utilizing the layered architectural approach automatically ensures a low coupling environment, predominantly relying exclusively on simple data coupling structures. The UI Presentation Layer is incapable of direct persistence access; it must communicate structurally through the Business Logic Layer. Furthermore, any volatile alterations originating from the OpenWeatherMap API solely influence the isolated Integration module. This protective buffering fundamentally prevents volatile networking changes from polluting or crashing the core database execution models, thereby granting extreme system flexibility and paving an unambiguous path toward future microservices migration if platform scaling demands necessitate such action.

7. UML Diagrams

7.1 Use Case Diagram

The Use Case Diagram visually describes the behavioral capabilities of the Digital Farmer Assistance System from the perspective of its primary actors. This diagram ensures that all high-level functional requirements defined in the SRS are adequately accounted for by corresponding system interactions.

Actors:

- **Farmer:** The primary end-user who consumes data and submits queries.
- **Admin (and Superadmin):** Personnel responsible for data entry, system modifications, user management, and expert query responses.

Primary Use Cases:

1. Farmer Interactions:

- *View Market Prices:* The farmer accesses a centralized dashboard displaying the latest commodity pricing (<<include>> Secure Login).
- *View Weather Information:* The farmer accesses five-day forecast data localized to their defined region (<<include>> Secure Login).
- *View Crop Advisory:* The farmer receives personalized data based on pre-registered crop types (<<include>> Secure Login).
- *Submit Query:* The farmer transmits textual questions directly to agricultural experts (<<include>> Secure Login).
- *Manage Profile:* The Farmer configures phone numbers, language preferences, and crop portfolios (<<include>> Secure Login).

2. Admin Interactions:

- *Manage Users:* Reviewing farmer registrations and editing permission structures.
- *Respond to Queries:* Administrating expert answers to farmer-submitted inquiries.

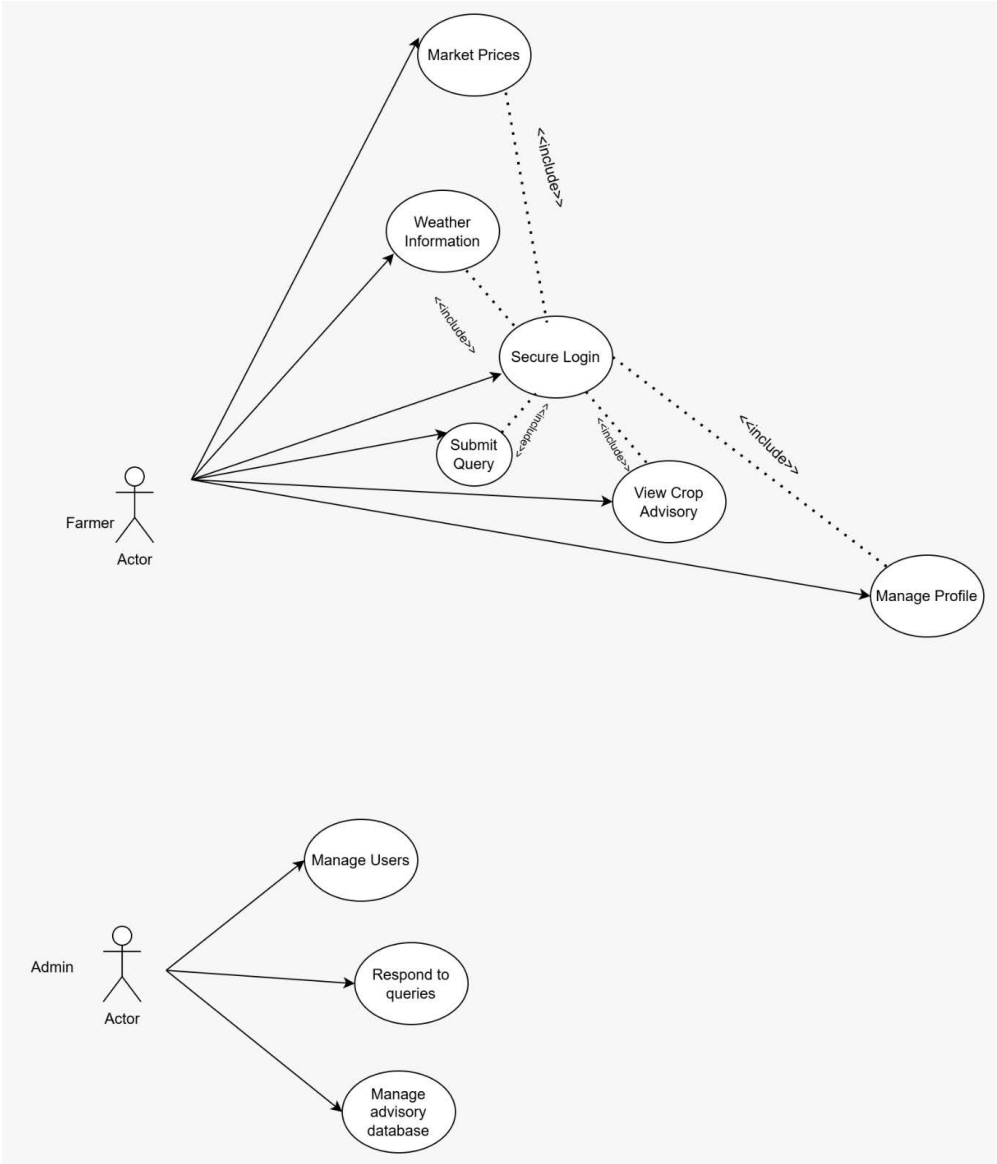


Figure 2: System Use Case Diagram

- *Manage Advisory Database*: Performing CRUD operations on crop knowledge base articles.
- *Manage Market Prices*: Entering newly verified commodity pricing metrics natively into the database.
- *Manage Weather Data*: Establishing manual weather overrides inside the database fallback system.

7.2 Class Diagram

The Class Diagram delineates the structural framework of the software, explicitly defining system objects, their attributes, behavioral methods, and their inter-relationships, thereby bridging the conceptual design into actionable object-oriented engineering strategies.

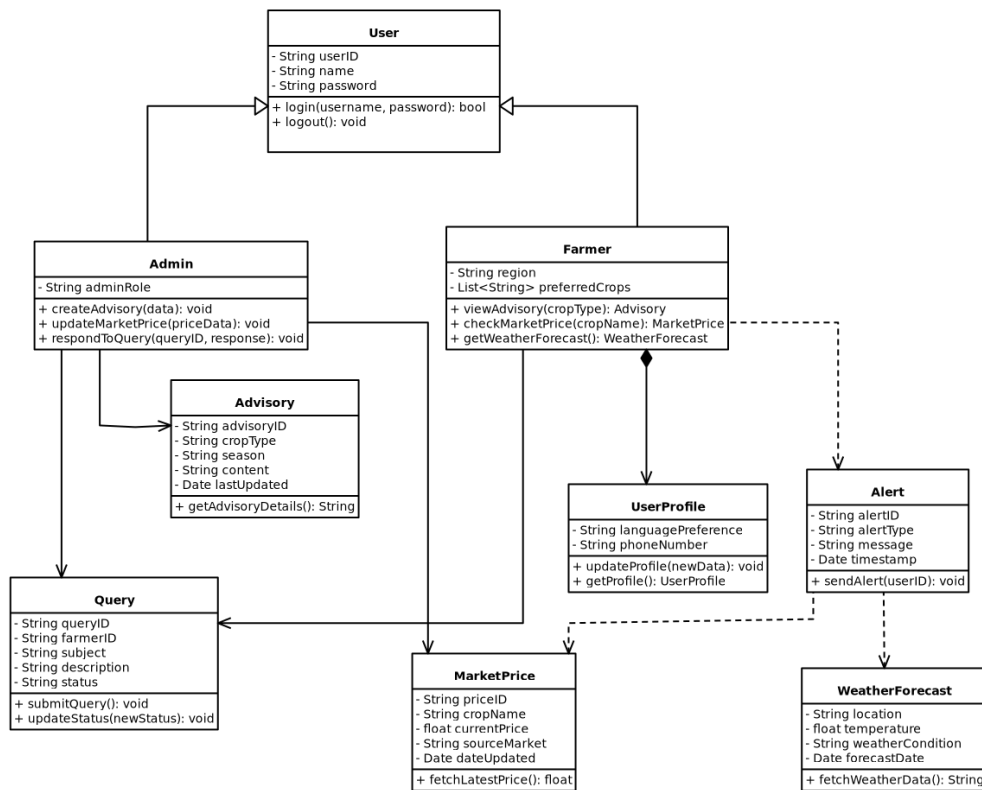


Figure 3: System Class Diagram

Core Class Definitions:

- **User (Abstract Base Class)**: Centralizes common authentication attributes (`userID`, `name`, `passwordHash`) and core methods (`login()`, `logout()`).
- **Admin**: Inherits `User`. Attributes include `adminRole`. Methods encompass authoritative interactions: `createAdvisory()`, `updateMarketPrice()`, and `respondToQuery()`.
- **Farmer**: Inherits `User`. Extends base class with `region` and `preferredCrops`. Specialized methodologies comprise `viewAdvisory()`, `checkMarketPrice()`, and `getWeatherForecast()`.

- **UserProfile:** Managed via composition by the Farmer class. Contains specific metadata including `languagePreference` and `phoneNumber`, manipulatable via `updateProfile()`.
- **Advisory:** Encapsulates structural data: `advisoryID`, `cropType`, `season`, `content`. Accessible via `getAdvisoryDetails()`.
- **Query:** Represents the interaction state: `queryID`, `subject`, `status`, and `response`, handled via `submitQuery()` and `updateStatus()`.
- **MarketPrice & WeatherForecast:** Abstraction layers handling external data synchronization, tracking `sourceMarket`, `currentPrice`, `temperature`, and `weatherCondition`.

Strategic Relationships: The software strictly employs hierarchical *inheritance* for shared user functionality (Farmer/Admin $\leftarrow$ User). It mandates strong *composition* mapping between the Farmer and their UserProfile (Farmer $\diamond \rightarrow$ UserProfile). Looser *dependency* and *association* relationships govern the dynamic interactions between the Farmer modules and rapidly volatile external data objects (Farmer $--- >$ MarketPrice).

8. Coding Strategy

8.1 Programming Language environment

The entirety of the Digital Farmer Assistance System is constructed utilizing JavaScript. The backend architecture dictates a dedicated Node.js (v18+) runtime environment. Node.js is selected explicitly for its non-blocking, asynchronous, event-driven infrastructure, which is universally recognized as highly optimal for I/O-heavy networking workloads—such as simultaneously processing dozens of concurrent MySQL database queries and external weather API connections without experiencing thread starvation. The frontend ecosystem leverages hyper-efficient Vanilla JavaScript (ES6+), HTML5, and CSS3. By completely excising heavy compilation steps or bulky client-side frameworks (e.g., React or Angular), the platform dramatically reduces initial payload sizes, creating a remarkably responsive interface optimally tailored for restrictive, low-bandwidth rural cellular environments. The persistence layer operates strictly on standard SQL protocols executing within a MySQL 8.0 cluster.

8.2 Tools Used

The system carefully curates a minimalist compilation of proven NPM libraries to accelerate development timelines while rigorously enforcing a minimal attack surface.

Table 3: Primary Third-Party Dependencies

Library Package	Architectural Purpose
Express.js	Provides the foundational HTTP routing server and essential middle-ware pipeline configurations.
mysql2	Provides an optimized MySQL connection pool natively supporting modern async/await syntax.
bcryptjs	Handles cryptographic password hashing protocols, utilizing a robust workload configuration of 10 salt rounds.
jsonwebtoken	Administers all stateless JSON Web Token generation and cryptographic validation algorithms.
dotenv	Enables rigorous environment variable management, cleanly separating credentials from the raw source code.
cors	Enforces Cross-Origin Resource Sharing security headers for strict browser-level access.

8.3 Project Architecture Structure

The file repository is rigorously structured to prioritize maintainability and an absolute separation of computing concerns. The hierarchy logically partitions raw configuration, middle-ware abstractions, and independent data route handlers into isolated, cohesive directories.

```

farmer-system/
|-- server.js           <- Express entry point
|-- db.js               <- MySQL connection pool
|-- schema.sql          <- Full database schema + seed data
|-- .env.example        <- Config template (copy to .env)
|-- package-lock.json
|-- package.json
|-- middleware/
|   |-- auth.js         <- JWT authentication middleware
|-- routes/
|   |-- auth.js         <- POST /api/auth/farmer/login, /register, /admin/login
|   |-- farmers.js      <- GET/PUT/DELETE /api/farmers
|   |-- market.js       <- GET/POST/PUT/DELETE /api/market
|   |-- weather.js      <- GET/POST/DELETE /api/weather (OWM API + DB fallback)
|   |-- advisory.js     <- GET/POST/PUT/DELETE /api/advisory
|   |-- queries.js      <- GET/POST/PUT/DELETE /api/queries
|   |-- admins.js       <- GET/POST/DELETE /api/admins
|-- public/
|   |-- index.html      <- Full frontend (HTML + CSS + JS)

```

8.4 Coding Standards

Codebase integrity is enforced via a rigid set of programmatic conventions:

- **RESTful Resource Protocols:** Endpoints strictly conform to HTTP verb definitions: GET for non-destructive reads, POST for record creation, PUT for total updates, and DELETE for sanitization.
- **Asynchronous Exception Handling:** Asynchronous operations mandate the deployment of encompassing try/catch structures, preventing fatal server termination upon localized execution failures.
- **Security Parameters:** All database interactions deploy aggressively parameterized SQL queries to categorically eradicate SQL injection vulnerabilities. Role-based Access Control (RBAC) validations are securely abstracted into middleware interceptors.

8.5 Representative Implementation Code

The following abbreviated code excerpt delineates the standard authentication pattern utilized throughout the system. The snippet powerfully demonstrates active SQL parameterization natively mitigating injection vectors, concurrent secure cryptographic comparison via bcrypt, and the final generation of an encrypted JSON Web Token for stateless access control.

```
1 router.post('/farmer/login', async (req, res) => {
2   const { phone, password } = req.body;
3   if (!phone || !password) return res.status(400).json({ error:
4     'phone_and_password_required' });
5
6   try {
7     const [rows] = await db.execute('SELECT * FROM FARMER WHERE
8       phone = ?', [phone]);
9     if (!rows.length) return res.status(401).json({ error: '
10       Invalid_credentials' });
11
12     const farmer = rows[0];
13     const match = await bcrypt.compare(password, farmer.
14       password);
15     if (!match) return res.status(401).json({ error: 'Invalid_
16       credentials' });
17
18     const token = jwt.sign(
19       { id: farmer.farmer_id, type: 'farmer', name: farmer.name
20         , region: farmer.region, lang: farmer.language_pref },
21       process.env.JWT_SECRET || 'secret', { expiresIn: '7d' }
22     );
```

```
17     res.json({ token, farmer: { farmer_id: farmer.farmer_id,
18         name: farmer.name, region: farmer.region, language_pref:
19         farmer.language_pref } });
20 } catch (err) {
21     res.status(500).json({ error: err.message });
22 }
```

Listing 1: Implementation of Farmer Authentication Protocol

9. Testing Strategy

9.1 Testing Approach

A comprehensive, multi-tiered testing strategy guarantees the absolute reliability and operational accuracy of the system architecture under varying load parameters. The testing protocol leverages three definitive evaluation paradigms:

1. **Unit Testing Framework:** Granular validation methodologies are deployed to scrutinize the behavioral accuracy of highly isolated functions—particularly cryptographic password hashing mechanisms and JSON web token string generation—independently from the broader application state.
2. **Integration Interfaces:** Rigorous evaluations analyze the operational handshakes between disparate architectural components. Crucially, integration testing ensures that API route handlers perform uninterrupted, synchronous data exchanges accurately bridging the Node.js application server, the local MySQL database persistence layer, and the external third-party OpenWeatherMap APIs.
3. **System and Usability Testing:** Extensive End-to-End (E2E) validations simulate a comprehensive farmer usage trajectory. Testing is heavily focused utilizing Chrome DevTools Device Mode to physically emulate low-resolution, mobile-viewport rendering, rigorously validating frontend HTML/CSS resilience and the accuracy of the multilingual interface.

9.2 Functional Test Cases

The core operational pathways governing CRUD capabilities and database interactions are validated against defined expected outcomes to actively verify systematic integrity.

Table 4: System Functional Test Cases

TC ID	Testing Input or Action	Expected Outcome	Status
TC01	Valid farmer phone string and secret password submitted via POST /api/auth/farmer/login	System returns HTTP 200 OK and a signed JWT bearer token.	Pass
TC02	Intentional submission of incorrect password parameters to the active login endpoint	System halts execution, returning HTTP 401 Unauthorized.	Pass
TC03	Authorized administrator email and password injected via POST /api/auth/admin/login	System validates and responds with a JWT imbued with an administrative role capability.	Pass
TC04	Unauthenticated GET /api/market request demanding global pricing arrays	HTTP 200 OK; a complete raw JSON array of recent crop prices is securely transmitted.	Pass
TC05	Appending ?crop=Rice query filter parameters onto a standard market GET string	HTTP 200 OK; the persistence layer surgically filters the JSON array exclusively outputting Rice data.	Pass
TC06	Construct valid REST POST /api/market request alongside an administrative JWT string	HTTP 201 Created; the database strictly commits the new pricing entry.	Pass
TC07	Unauthorized Farmer attempts to inject a POST /api/market execution command	The middleware interceptor rapidly isolates the violation, returning HTTP 403 Forbidden.	Pass
TC08	Client fires GET /api/weather?locations=Palakkad	HTTP 200 OK; live API or offline database returns relevant weather matrices.	Pass
TC09	Interrogate the system alerts channel via GET /api/weather/alerts	HTTP 200 OK; application identifies and parses any active location tagged with is_extreme.	Pass
TC10	Transmit GET /api/advisory?crop=Rice&season=Kharif structural queries	HTTP 200 OK; the database applies bidirectional filtering to extract specific operational advice.	Pass
TC11	Farmer fires GET /api/advisory/for-farmer carrying an authenticated token	HTTP 200 OK; logic seamlessly extracts user profile preferences, yielding bespoke data without manual filters.	Pass
TC12	Initiating a POST /api/queries workflow while transmitting a farmer JWT	HTTP 201 Created; a relational query record is initialized under an explicitly pending status.	Pass
TC13	Administrator executes a PUT /api/queries/:id/respond command bridging an answer string	HTTP 200 OK; the database securely updates the targeted query ticket with the expert answer.	Pass

Continued on next page

Table 4 – *Continued from previous page*

TC ID	Testing Input or Action	Expected Outcome	Status
TC14	Initiating a GET <code>/api/farmers/me</code> using an established farmer JWT authentication string	HTTP 200 OK; the persistence tier locates and serves the specific localized user profile arrays.	Pass
TC15	Initiating an authorized dashboard GET <code>/api/stats</code> action sequence	HTTP 200 OK; comprehensive database summarizations, counters, and extreme alert indices are provided.	Pass

9.3 Boundary and Negative Test Cases

Negative testing aggressively challenges the software paradigm, assessing whether the logic accurately catches, prevents, or bypasses intentional structural violations or deliberate resource deprivations without triggering fatal system crashes.

Table 5: Boundary and Negative Scenario Emulations

TC ID	Scenario Input Description	Expected Structural Outcome	Result
TC16	Request submitted targeted closely at a protected routing endpoint totally lacking an HTTP Authorization bearer token string.	The core router blocks progress, universally returning HTTP 401 Unauthorized.	Pass
TC17	Deliberately trigger weather data compilation with a malformed or omitted external API key.	The connection dynamically fails over to the persistence layer, rendering offline database fallback data via HTTP 200.	Pass
TC18	User executes a GET command targeting <code>/api/nonexistent</code> structural paths.	Unroutable parameters terminate gracefully; the system replies strictly returning a generic HTTP 404 Not Found error.	Pass
TC19	Command dispatched to POST <code>/api/auth/farmer/register</code> forcefully injecting an identical phone number already mapped.	Relational rules forbid duplication; handler accurately triggers an HTTP 409 Conflict protocol.	Pass
TC20	Basic-tier Farmer attempts to utilize a valid token to perform POST <code>/api/admins</code> .	Authorization interceptors actively analyze the token hierarchy, preventing administrative escalation by returning HTTP 403.	Pass

10. Deployment Plan

10.1 Deployment Environment

Transitioning the Digital Farmer Assistance System from a local development environment to a robust production infrastructure assumes several baseline environmental prerequisites to maintain continuous operational stability.

- **Operating System:** A stable Linux distribution (preferably Ubuntu 22.04 LTS or newer) is strongly recommended for the deployment staging environment, providing optimal security patching protocols.
- **Runtime Configuration:** The host server must possess a dedicated installation of Node.js v18 LTS (or greater).
- **Database Architecture:** The persistence layer utilizes MySQL 8.0+. While a native local instance is permissible for prototyping, a managed cloud database solution (e.g., PlanetScale or AWS RDS) is aggressively recommended for production environments to handle automated backups.
- **Web Server / Reverse Proxy:** The native Express.js process exclusively handles HTTP routing and serves static layout files from the `public/` directory. For production-tier deployment, an Nginx reverse proxy layer is strictly advised to correctly terminate SSL certificates via certbot and port-forward requests securely.

10.2 Local Deployment Steps

Initializing a development or localized testing environment requires the sequential execution of the following rigid configurational mandates:

1. **Prerequisite Acquisition:** Ensure comprehensive installations of Node.js v18+ and MySQL 8.0 exist on the local workstation.
2. **Repository Cloning:** Extract or clone the project source files into a securely designated directory and navigate into the project root via terminal. Execute `npm install` to rapidly resolve all NPM package dependencies listed in `package.json`.
3. **Environment Contextualization:** Copy the standardized `.env.example` template via terminal command `cp .env.example .env`. Securely populate the newly generated `.env` file configurations, specifically overriding `DB_PASSWORD`, `JWT_SECRET`, `PORT`, and the external `WEATHER_API_KEY`.
4. **Schema Provisioning:** Initialize the raw database schema and seed demonstration entities by executing: `mysql -u root -p < schema.sql`.
5. **Server Ignition:** Launch the primary Node.js thread by executing `node server.js`. The server will output a successful connection heuristic console string.
6. **Client Access:** Navigate any standard web browser cleanly to `http://localhost:3000` to render the frontend portal interface.

10.3 Cloud Deployment Implementation

Deploying the platform for continuous, broad internet accessibility involves abstracting local prerequisites into managed cloud architectures.

- **PaaS Platform Selection:** Low-friction Platform as a Service (PaaS) providers—such as Railway.app, Render.com, or Fly.io—are targeted. These services natively incorporate Node.js containerization.

- **Secrets Management:** Raw configuration parameters natively stored in `.env` files locally are meticulously transcribed directly into the selected cloud provider's encrypted environment variable dashboard to ensure rigid repository security.
- **Managed Database Strategies:** The schema persistence tier is dynamically hosted using managed Database-as-a-Service providers like AWS RDS Free Tier or direct planetScale integrations to assure continuous uptime.

10.4 Ongoing System Maintenance

Sustained operability dictates rigorous application maintenance post-deployment.

- **Cryptographic Rotation:** The `JWT_SECRET` key will undergo periodic cyclical rotation. Updating the environment variable and cycling the node service automatically invalidates old tokens forcing a security login refresh.
- **API Key Monitoring:** The external OpenWeatherMap API key (capped at the free tier of 60 calls/min) must be continually monitored via provider dashboards to detect latency throttling heuristics.
- **Data Integrity:** Standardized database backups shall be scheduled via external `mysqldump` cron jobs protecting against catastrophic persistence failure.

11. Reflection and Learning Outcomes

11.1 Challenges faced during client interaction

The conception and execution of the Digital Farmer Assistance System introduced several technical and structural challenges requiring meticulous engineering solutions.

- **Resolving Requirements Ambiguities:** The initial problem statement possessed significant ambiguities surrounding the subjective definition of "timely access" and the physical geographic boundaries dictating regional market scopes. Significant time was expended systematically translating undefined variables into strictly measurable operational metrics.
- **Offline Durability vs. Live Data:** A fundamental tension existed between capturing highly volatile, real-time external API meteorological data and acknowledging the harsh reality of an inherently unstable internet connectivity ecosystem in rural India. Designing a seamless, transparent database fallback caching mechanism that bypassed API failures without crashing the UI proved technically demanding.
- **Authorization Scoping:** Constructing a strictly rigid role-based access control paradigm separating general Farmers from highly privileged Administrators (and underlying Superadmins) necessitated complex JSON Web Token payload structuring. Establishing routing middleware capable of actively reading token roles and rejecting unauthorized execution paths required careful debugging.
- **Database-Level Localization:** Developing a dynamic multilingual capability strictly using database language attribute flags—purposefully omitting bloated application-

layer localization libraries (like i18n)—required heavily indexing specific querying routines to ensure the system efficiently filtered distinct linguistic payloads seamlessly based on farmer profile preferences.

- **Mobile-First Simplicity:** Designing an intrinsically simplistic HTML UI optimized aggressively for populations demonstrating low digital literacy forced a radical simplification of traditional web patterns, omitting complex JavaScript layouts in favor of raw, high-contrast, linear dashboard cards.

11.2 Lessons learned

The engineering lifecycle continuously underscored vital software engineering dictums.

- **Rework Mitigation:** Early, aggressive requirements clarification unambiguously prevented mathematically expensive codebase rewrites by firmly establishing the structural scope prior to executing a single line of backend logic.
- **Architectural Separation:** Committing rigorously to a four-layer software architecture unambiguously simplified complex unit testing processes and allowed UI alterations completely independent of structural database operations.
- **Parametrized Security:** Hard-coding parameterized SQL variables across all structural queries is utterly non-negotiable; ignoring strict data sanitization inevitably exposes the entire platform to trivial SQL injection compromise parameters.
- **UML Efficacy:** Utilizing abstract Use Case and Class diagrams effectively translated isolated technical schemas into easily digestible communicable data points understandable globally by the development team.

12. References

References

- [1] Pressman, R.S. and Maxim, B.R., *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill Education, 2020.
- [2] Sommerville, I., *Software Engineering*, 10th ed., Pearson Education, 2016.
- [3] OpenWeatherMap, “API Documentation,” <https://openweathermap.org/api>. [Online]. Accessed: 2024.
- [4] OpenJS Foundation, “Express.js Documentation,” <https://expressjs.com>. [Online]. Accessed: 2024.
- [5] Oracle Corporation, “MySQL 8.0 Reference Manual,” <https://dev.mysql.com/doc/>. [Online]. Accessed: 2024.
- [6] Auth0, “Introduction to JSON Web Tokens,” <https://jwt.io/introduction>. [Online]. Accessed: 2024.
- [7] United Nations, “Sustainable Development Goals,” <https://sdgs.un.org/goals>. [Online]. Accessed: 2024.
- [8] Fowler, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

13. Appendix A – Product Screenshots

This appendix presents visual representations of the Digital Farmer Assistance System interface and backend architecture. These screenshots demonstrate the core functionalities implemented to fulfill the project's requirements.

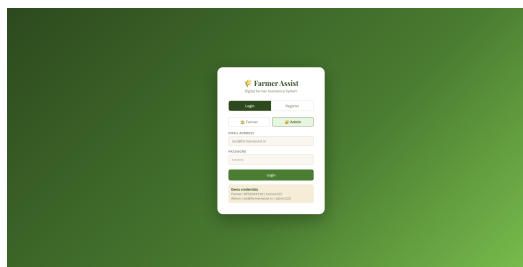


Figure 4: Farmer Login Interface: Mobile-first phone number and password authentication form.

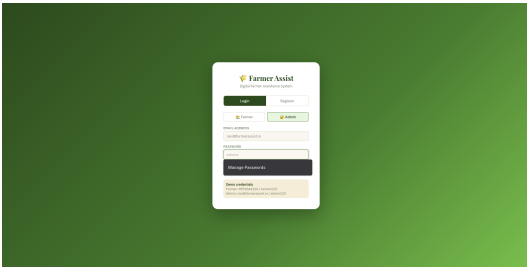


Figure 5: Admin Login Interface: Email and password form mapped exclusively for administrator user access tracking.

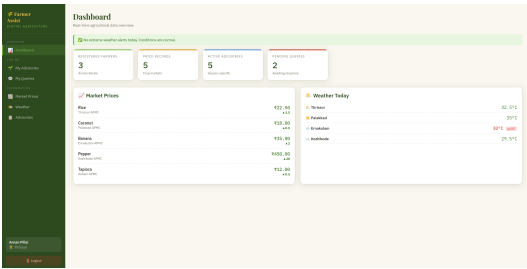


Figure 6: Farmer Dashboard: Unified data stream rendering specific crop advisories, an integrated 5-day weather forecast, and local market price summary cards.

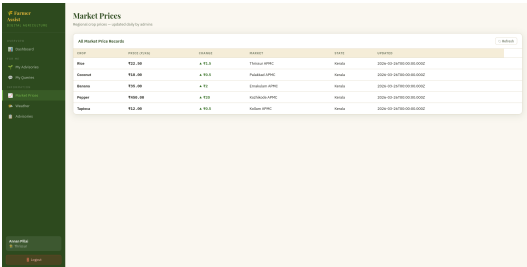


Figure 7: Market Prices Module: Searchable and highly filterable data table rendering recent crop prices tagged with the active date and source market coordinates.

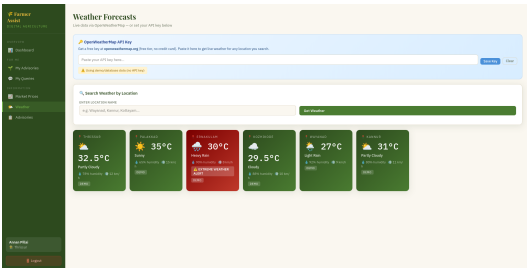


Figure 8: Weather Module: Rendered 5-day localized weather sequence showcasing the automated alert banners dynamically triggered by external extreme weather metric flags.

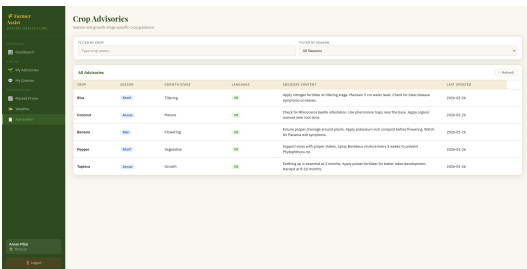


Figure 9: Crop Advisory Module: Rendered layout featuring season-filtered advisory content dynamically leveraging the application’s native multilingual database translation labels.

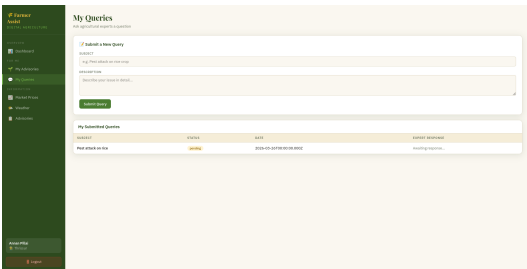


Figure 10: Query Submission Form: Interactive module utilized by farmers to directly submit agricultural issue reports explicitly requesting expert administrative resolutions.

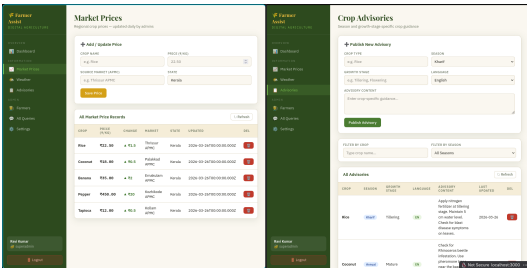


Figure 11: Admin Panel Architecture: Backend CRUD (Create, Read, Update, Delete) portal exclusively used by administrative profiles to constantly manage advisory flows and market price points.